

STATIC INTEGRATION REVIEW

LiftUp SaaS Full-Stack Audit Report

Comprehensive analysis of API contracts, state management, database integrity, and concurrency safety across the multi-tenant AI reservation platform.

Platform LiftUp SaaS v2.0

Scope Backend + Frontend Integration

Rating 6.5 / 10

Date June 2, 2026

Table of Contents

1. Executive Summary	2
2. API Contract Verification	3
2.1 Request Header Analysis	3
2.2 Request Body Contract	3
2.3 Response Contract	3
2.4 Error Response Handling	4
3. State Management and Flow Analysis	4
3.1 Session Token Lifecycle	4
3.2 Complete Reservation Flow Trace	5
3.3 Critical: modify_reservation Split-Transaction Race	6
3.4 Confirmation Code Generation Vulnerability	6
4. Automated Testing Suite	7
5. Executive Feedback Report	9
5.1 Overall Integration Rating: 6.5 / 10	9
5.2 Top 3 Biggest Risks	9
5.3 Complete Findings Summary	10
6. Prioritized Recommendations	11

1. Executive Summary

This report presents the findings of a comprehensive static integration review of the LiftUp SaaS multi-tenant AI reservation platform. The review examined every layer of the full-stack application, from the React frontend testing dashboard through the FastAPI middleware chain, Pydantic validation, AI agent tool executor, async database layer, and PostgreSQL schema with Row-Level Security. The objective was to identify integration bugs, contract mismatches, state management flaws, and concurrency risks that would manifest under production load with 50 or more concurrent restaurant tenants.

The overall integration quality score is **6.5 / 10**. The backend demonstrates sophisticated multi-tenant isolation with RLS, quota enforcement, and tenant-scoped transactions. However, several critical issues were identified: a confirmation code generation vulnerability using non-cryptographic randomness, a split-transaction race condition in the reservation modification flow, a subtle header transmission risk for session tokens, and an end-to-end encryption gap for API keys in transit. The frontend dashboard is well-structured and correctly implements the chat API contract, but the session token lifecycle has a subtle edge case where a stale token could break conversation continuity.

Category	Score	Assessment
API Contract Fidelity	8 / 10	Headers and payloads match correctly; minor session_token edge case
State Management	6 / 10	Session continuity works but modify_reservation has a TOCTOU race
Database Integrity	7 / 10	RLS and tenant isolation are strong; confirmation code is weak
Concurrency Safety	5 / 10	SELECT FOR UPDATE NOWAIT is good but modify splits transactions
Security Posture	6 / 10	Good middleware chain; API keys lack TLS enforcement in code
Frontend Integration	8 / 10	Clean contract implementation; celebration flow is correct
OVERALL	6.5 / 10	Not production-ready for 50 tenants without fixes

2. API Contract Verification

This section performs a line-by-line comparison of the frontend fetch calls against the FastAPI route handlers and Pydantic models for the primary integration endpoint: POST /api/widget/chat. Every header, every JSON field, and every response property was traced from the React component through the network layer to the Python handler and back.

2.1 Request Header Analysis

The frontend (page.tsx, line 283-287) constructs the following headers for each chat request:

```
const requestHeaders: Record<string, string> = { "Content-Type":  
  "application/json", "X-API-Key": apiKey.trim(), "X-Session-Token": sessionToken,  
};
```

The backend CORS middleware (main.py, line 176-183) explicitly allows these headers:

```
allow_headers=["Content-Type", "X-API-Key", "X-Session-Token"]
```

The tenant authentication middleware (main.py, line 207) reads X-API-Key, and the session dependency (main.py, line 337) reads X-Session-Token. **Verdict: All required headers are correctly sent and received. The contract is fully satisfied.**

FINDING [LOW]: The X-Session-Token header is listed in CORS `expose_headers` (main.py line 183), meaning the server intends to return it as a response header. However, the frontend reads `session_token` from the JSON response body (`data.session_token`), not from the response header. This is not a bug because the `ChatResponse` Pydantic model includes `session_token` as a body field, but the `expose_headers` directive is redundant and could confuse future developers.

2.2 Request Body Contract

The frontend sends: { "message": "user text here" }. The backend `ChatRequest` Pydantic model (main.py, line 305-311) expects a "message" field (required, `min_length=1`, `max_length=2000`) and an optional "session_token" field. The frontend does not send `session_token` in the body because it is transmitted via the X-Session-Token header instead. The backend correctly reads the session token from the header in the `get_session` dependency (line 337-338), not from the body. This is consistent.

Verdict: The JSON payload matches perfectly. The body field "session_token" in ChatRequest is declared Optional and defaults to None, so omitting it is valid. The backend never reads it from the body in the actual handler flow.

2.3 Response Contract

The `ChatResponse` Pydantic model (main.py, line 314-318) returns four fields. The frontend correctly handles all of them:

reply (str, required)	Frontend reads data.reply and displays it as the assistant message (page.tsx line 339). If empty, falls back to "(empty reply)". Correct.
session_token (str, required)	Frontend reads data.session_token and calls setSessionToken() (line 332-334). This updates the React state so the next request uses the new token. Correct.
booking_complete (bool, default=False)	Frontend reads data.booking_complete (line 341). If true AND confirmation_code exists, triggers the celebration overlay. Correct.
confirmation_code (str None)	Frontend reads data.confirmation_code (line 342). Displayed in both the chat bubble and the celebration modal. Correct.

FINDING [MEDIUM]: The backend always returns session_token in the response body, but the frontend also sends it as a request header. If the server generates a new token (e.g., when the existing one is invalid or expired), the frontend correctly updates its state. However, there is a subtle race condition: if two rapid messages are sent before the first response returns, both requests carry the SAME session token. The second response may return a different session_token, overwriting the first. Since the isLoading flag prevents concurrent sends (the submit button is disabled), this is unlikely in practice but not architecturally impossible (e.g., if a user triggers sendMessage programmatically).

2.4 Error Response Handling

The frontend error handling (page.tsx line 318-329) covers two scenarios: HTTP error responses and network failures. For HTTP errors, it attempts to extract data.detail.message or data.detail, then falls back to a generic HTTP status message. This correctly handles the backend error format, which returns {"detail": "message"} for 401/402 errors and {"detail": {"code": "...", "message": "..."}} for 402 quota-exceeded responses. The network error handler displays a clear message asking if the backend is running. **Verdict: Error contract is well-handled.**

3. State Management and Flow Analysis

This section traces the full lifecycle of a reservation from the initial user message through the AI agent conversation, tool execution, database persistence, and back to the frontend UI. It examines how session tokens maintain conversation memory and identifies edge cases where the UI could show a success message that does not reflect the actual database state.

3.1 Session Token Lifecycle

The session token is the linchpin of conversation continuity. Here is the complete lifecycle as it flows through the system:

1. Frontend Generation: On page load, `generateSessionToken()` (page.tsx line 58-64) creates a 32-character string using `crypto.getRandomValues()`. This is cryptographically secure and sufficient for session identification. The user can override this token to test returning customer scenarios.

2. Backend Lookup: The `get_session` dependency (main.py line 336-345) receives the token from the `X-Session-Token` header. If the token is missing or shorter than 16 characters, it generates a new one using `secrets.token_urlsafe(32)`. It then calls `get_or_create_session` (database.py line 690-724), which looks up an existing active session or creates a new one. The session includes the conversation history (messages JSONB) and booking context (context JSONB).

3. Conversation Memory: On each request, the session messages are loaded into `AgentContext` (main.py line 377), which is passed to `run_agent()`. The agent appends the user message to the history, calls Claude with the full conversation, and the assistant response (including tool calls and results) is appended. After the agent completes, `update_session` (database.py line 727-753) persists the updated messages and context back to PostgreSQL.

4. Token Return: The server returns the `session_token` in the `ChatResponse` body. The frontend updates its state (page.tsx line 332-334), so subsequent requests carry the same token. This ensures that multi-turn conversations maintain continuity across browser refreshes if the user copies the token back, but the token is stored only in React state (not `localStorage`), so a page refresh creates a new session.

FINDING [MEDIUM]: The frontend does not persist the session token in `localStorage` or `sessionStorage`. A page refresh generates a brand-new session, losing all conversation history. For a testing dashboard, this is acceptable behavior, but it means the "returning customer" test scenario requires manually pasting the token. More critically, the backend sessions expire after 2 hours (database.py line 338), but the frontend has no awareness of this expiry. If a user leaves the dashboard open for more than 2 hours and resumes chatting, the backend will silently create a new session, but the frontend will continue displaying the old conversation history in the UI alongside the new session responses, creating a confusing experience where Aria seems to have "forgotten" the earlier context.

3.2 Complete Reservation Flow Trace

A successful reservation flows through the following stages. Each stage was traced through the actual code to identify potential failure points:

Stage 1 - Chat Request: User types a message, frontend sends `POST /api/widget/chat` with `X-API-Key` and `X-Session-Token`. `TenantAuthMiddleware` validates the API key and loads the restaurant record. The `enforce_quota` dependency checks billing status. If either fails, a 401 or 402 is returned and the frontend displays the error. No database writes have occurred yet, so there is no inconsistency risk.

Stage 2 - Agent Execution: The agent builds a dynamic system prompt from the restaurant profile, calls Claude 3.5 Sonnet with tool definitions, and Claude responds with tool calls (e.g., `check_table_availability`, `find_or_create_customer`, `book_table`). Each tool call is executed by `execute_tool` (agent.py line 364-506), which injects `restaurant_id` from the server-side context (never from Claude output). This is a strong security design.

Stage 3 - Table Booking: The `book_table` function (database.py line 489-584) runs inside a `tenant_transaction`, which sets the RLS variable and wraps everything in an atomic transaction. It locks the table row with `SELECT FOR UPDATE NOWAIT`, re-verifies no time overlap, generates a confirmation code, inserts the reservation, and increments the customer visit count. If any step fails, the entire transaction rolls back.

Stage 4 - Response: The `ChatResponse` includes `booking_complete: true` and the `confirmation_code`. The frontend displays the code in the chat bubble and triggers the celebration overlay. At this point, the database transaction has committed, so the UI state matches the database state. **For the `book_table` path, there is no rollback risk.**

3.3 Critical: modify_reservation Split-Transaction Race

FINDING [CRITICAL]: The `modify_reservation` function (database.py line 638-683) performs a `cancel` and `rebook` across TWO SEPARATE TRANSACTIONS, creating a Time-of-Check-to-Time-of-Use (TOCTOU) race condition that can cause data loss.

The flow is: (1) `get_reservation_by_code` runs in one transaction, (2) `check_availability` runs in another transaction, (3) the old reservation is cancelled in a third transaction (line 669-673), and (4) `book_table` runs in a fourth transaction (line 675-683). Between step 3 (cancel) and step 4 (rebook), the time slot is momentarily free. If another concurrent request books that same slot, the rebook in step 4 will fail with a `ValueError` from `book_table`. The user sees an error from the AI agent, but their original reservation has already been cancelled. The data is lost with no rollback.

Furthermore, the `cancel` in step 3 uses a separate `tenant_transaction` (database.py line 668-673), so its `COMMIT` is independent of the `book_table` transaction. There is no way to atomically cancel the old reservation and create the new one. The correct fix is to perform both the cancellation and the new booking inside a single database transaction.

3.4 Confirmation Code Generation Vulnerability

FINDING [HIGH]: The `_make_confirmation_code` function (database.py line 482-486) uses Python's `random.choices()` instead of `secrets.choice()` for generating confirmation codes. The first line calls `secrets.choice()` correctly, but the result is immediately discarded on line 485 where `random.choices()` overwrites it.

```
def _make_confirmation_code() -> str: alphabet = string.ascii_uppercase +
string.digits code = "".join(secrets.choice(alphabet) for _ in range(6)) # line
484: SECURE but unused chars = random.choices(string.ascii_uppercase +
```

```
string.digits, k=6) # line 485: INSECURE return "RES-" + "".join(chars) # line
486: uses INSECURE chars
```

Python's random module uses the Mersenne Twister PRNG, which is not cryptographically secure. An attacker who observes a modest number of confirmation codes can predict future codes, enabling unauthorized reservation cancellations or modifications. The fix is trivial: replace random.choices() with a secrets-based generator and remove the dead code on line 484.

4. Automated Testing Suite

Below is a complete pytest script using httpx.AsyncClient to simulate 5 concurrent users hitting the POST /api/widget/chat endpoint simultaneously. The test is designed to exercise the database race-condition locks (SELECT FOR UPDATE NOWAIT) and detect double-booking scenarios. The script requires the backend to be running at http://127.0.0.1:8000 with a valid restaurant API key configured.

```
""" test_concurrent_chat.py – Race condition test suite for LiftUp SaaS Run:
pytest test_concurrent_chat.py -v --tb=long Requires: httpx, pytest,
pytest-asyncio """ import asyncio import pytest import httpx import time import
uuid BASE_URL = "http://127.0.0.1:8000" API_KEY = "YOUR_TEST_API_KEY" # Replace
with real key NUM_CONCURRENT_USERS = 5 SESSION_TOKENS =
[f"test-session-{uuid.uuid4().hex}" for _ in range(NUM_CONCURRENT_USERS)]
@pytest.fixture def headers_factory(): """Returns a factory that creates headers
for a given user index.""" def _make(idx: int) -> dict: return { "Content-Type":
"application/json", "X-API-Key": API_KEY, "X-Session-Token": SESSION_TOKENS[idx],
} return _make @pytest.mark.asyncio async def test_health_check(): """Verify the
backend is reachable before running load tests.""" async with
httpx.AsyncClient(base_url=BASE_URL, timeout=10) as client: resp = await
client.get("/health") assert resp.status_code == 200 data = resp.json() assert
data.get("status") == "healthy" @pytest.mark.asyncio async def
test_five_concurrent_chat_sessions(headers_factory): """Simulate 5 users sending
chat messages simultaneously. This tests that: 1. Each session gets its own
conversation context 2. Session tokens do not leak across tenants 3. The
middleware correctly routes each request """ async def send_message(idx: int,
message: str) -> httpx.Response: async with httpx.AsyncClient(base_url=BASE_URL,
timeout=30) as client: return await client.post( "/api/widget/chat",
headers=headers_factory(idx), json={"message": message}, ) messages = [ "I'd like
a table for 2 tonight", "Can I book for 4 people tomorrow at 7pm?", "Do you have
outdoor seating available?", "Table for 6 this Friday at 8pm please", "I need to
cancel my reservation", ] # Fire all 5 requests concurrently tasks =
[send_message(i, messages[i]) for i in range(NUM_CONCURRENT_USERS)] responses =
await asyncio.gather(*tasks, return_exceptions=True) # Verify all requests
succeeded (or got expected business errors) for idx, resp in
enumerate(responses): if isinstance(resp, Exception): # Network error -
acceptable if backend is not running pytest.skip(f"User {idx}: Network error -
{resp}") assert resp.status_code in (200, 401, 402, 429), ( f"User {idx}:
Unexpected status {resp.status_code}: " f"{resp.text[:200]}" ) if
resp.status_code == 200: data = resp.json() assert "reply" in data, f"User {idx}:
Missing 'reply' in response" assert "session_token" in data, f"User {idx}:"
```



```

Missing 'session_token'" @pytest.mark.asyncio async def
test_booking_race_condition(headers_factory): """Test that two users cannot
double-book the same table slot. Strategy: Two users attempt to book the same
date/time/party_size simultaneously. One should succeed; the other should get a
conflict error from the AI agent (table just booked). This directly tests the
SELECT FOR UPDATE NOWAIT lock in book_table(). """ # Step 1: Both users ask for
availability (sequential, to get context) async with
httpx.AsyncClient(base_url=BASE_URL, timeout=30) as client: # Warm up both
sessions with a greeting for idx in range(2): await client.post(
"/api/widget/chat", headers=headers_factory(idx), json={"message": "Hello"}, ) #
Step 2: Both users simultaneously request the same slot booking_msg = "I'd like
to book a table for 4 people on 2026-07-15 at 19:00" async def book_slot(idx:
int) -> httpx.Response: async with httpx.AsyncClient(base_url=BASE_URL,
timeout=60) as client: return await client.post( "/api/widget/chat",
headers=headers_factory(idx), json={"message": booking_msg}, ) responses = await
asyncio.gather( book_slot(0), book_slot(1), return_exceptions=True ) successes =
0 for resp in responses: if isinstance(resp, Exception): continue if
resp.status_code == 200: data = resp.json() if data.get("booking_complete"):
successes += 1 # At most ONE user should successfully book the same slot assert
successes <= 1, ( f"RACE CONDITION DETECTED: {successes} users booked the same
slot!" ) @pytest.mark.asyncio async def test_session_isolation(headers_factory):
"""Verify that conversation sessions do not leak between users.""" unique_msgs =
[f"Secret word: {uuid.uuid4().hex[:8]}" for _ in range(2)] async with
httpx.AsyncClient(base_url=BASE_URL, timeout=30) as client: # User 0 sends a
secret message resp0 = await client.post( "/api/widget/chat",
headers=headers_factory(0), json={"message": unique_msgs[0]}, ) assert
resp0.status_code == 200 # User 1 asks "what did I just say?" resp1 = await
client.post( "/api/widget/chat", headers=headers_factory(1), json={"message":
"What was the secret word I mentioned?"}, ) assert resp1.status_code == 200 data1
= resp1.json() # User 1 should NOT see User 0's secret word assert unique_msgs[0]
not in data1.get("reply", ""), ( f"SESSION LEAK: User 1 can see User 0's
message!" ) @pytest.mark.asyncio async def test_rate_limiting(headers_factory):
"""Verify that rate limiting (30/min) works correctly.""" async with
httpx.AsyncClient(base_url=BASE_URL, timeout=10) as client: # Send 35 rapid
requests (limit is 30/min) responses = [] for _ in range(35): try: resp = await
client.post( "/api/widget/chat", headers=headers_factory(0), json={"message":
"hi"}, ) responses.append(resp.status_code) except httpx.HTTPStatusError as e:
responses.append(e.response.status_code) # At least some should be rate-limited
(429) rate_limited = sum(1 for s in responses if s == 429) assert rate_limited >
0, ( f"Rate limiting not working: 0/35 requests were limited" )

```

This test suite covers four critical scenarios: (1) basic health check to ensure the backend is reachable, (2) five concurrent users hitting the chat endpoint simultaneously to verify session isolation and middleware correctness, (3) a deliberate race condition test where two users attempt to book the same time slot simultaneously, directly exercising the SELECT FOR UPDATE NOWAIT lock, and (4) session isolation verification ensuring conversation history does not leak between users. The rate limiting test verifies that slowapi enforces the 30/minute constraint. Run the suite with "pytest test_concurrent_chat.py -v --tb=long" after replacing YOUR_TEST_API_KEY with a valid key.

5. Executive Feedback Report

This section provides a direct, unvarnished assessment of the LiftUp SaaS platform's readiness for a 50-restaurant production launch. The evaluation is based on the complete codebase review spanning all seven source files (`main.py`, `agent.py`, `database.py`, `notifications.py`, `onboarding.py`, `config.py`, `schema.sql`) and the React frontend testing dashboard.

5.1 Overall Integration Rating: 6.5 / 10

The LiftUp platform demonstrates a strong architectural foundation with multi-tenant isolation at both the application level (`restaurant_id` in every `WHERE` clause) and the database level (RLS policies with session variables). The dynamic AI prompt system is elegant, and the security-first approach of stripping `restaurant_id` from Claude's tool inputs is commendable. However, several production-blocking issues prevent a higher score. The split-transaction race condition in `modify_reservation` is the most severe, as it can cause data loss under concurrent load. The confirmation code vulnerability is a security issue that is trivially exploitable. And the lack of TLS enforcement for API keys in transit means the system relies entirely on deployment-level configuration for encryption, with no application-level guarantee.

5.2 Top 3 Biggest Risks

Risk #1: `modify_reservation` Split-Transaction Data Loss (CRITICAL)

The `modify_reservation` function cancels the existing reservation in one transaction and creates a new one in a separate transaction. Between these two commits, the time slot is free, and any concurrent booking can claim it. If the rebook fails (due to a race), the user's original reservation is already cancelled with no way to recover. In a 50-restaurant production environment with multiple agents processing bookings simultaneously, this WILL happen. The probability increases with traffic volume, and the impact is severe: a confirmed reservation disappears with no audit trail to restore it. The fix is to perform both the cancellation and the new booking within a single database transaction, which requires refactoring the `modify_reservation` function to inline the `book_table` logic or to use a savepoint pattern within one transaction scope.

Risk #2: Confirmation Code Predictability (HIGH)

The use of `random.choices()` instead of `secrets.choice()` for confirmation code generation means codes are generated by the Mersenne Twister PRNG, which is deterministic and predictable given 624 observed outputs. An attacker who collects confirmation codes from booking confirmations (visible in emails or SMS) can predict future codes and use them to cancel or modify other guests' reservations. This is not a theoretical risk; the Mersenne Twister state can be recovered from just 624 32-bit outputs, and each confirmation code provides 6

base-36 characters (approximately 31 bits). With approximately 20 observed codes, an attacker can reconstruct the PRNG state and predict all future codes. The fix is a single-line change: replace `random.choices()` with `[secrets.choice(alphabet) for _ in range(6)]` and remove the dead code that generates the unused "code" variable.

Risk #3: No TLS Enforcement for API Keys in Transit (HIGH)

The X-API-Key is transmitted as a plain HTTP header on every request. The CORS configuration allows `origins=["*"]`, meaning the widget can be embedded on any domain, including those served over HTTP rather than HTTPS. If a restaurant's website does not use HTTPS (which is still common for small businesses), the API key is transmitted in cleartext and can be intercepted by any network observer. The code has no mechanism to enforce HTTPS or reject non-TLS requests. The `config.py` `PLATFORM_URL` defaults to `"https://api.liftupai.com"`, but this is only used for widget URLs and billing redirects, not for enforcing transport security on API requests. The fix should include middleware that rejects requests without a valid `X-Forwarded-Proto: https` header when running behind a reverse proxy, and the CORS configuration should be tightened to specific domains rather than allowing all origins.

5.3 Complete Findings Summary

Severity	Location	Description
CRITICAL	database.py:638-683	<code>modify_reservation</code> splits cancel+rebook across two independent transactions. Between the cancel COMMIT and the rebook, a concurrent booking can steal the slot, causing data loss with no recovery path.
HIGH	database.py:485	Confirmation code uses <code>random.choices()</code> (Mersenne Twister, deterministic) instead of <code>secrets.choice()</code> (cryptographic). The secure code on line 484 is dead code, never used.
HIGH	main.py:178	CORS <code>allow_origins=["*"]</code> allows API key transmission from any domain, including HTTP origins. No TLS enforcement exists at the application level.
MEDIUM	page.tsx:225-227	Session token not persisted in <code>localStorage/sessionStorage</code> . Page refresh creates a new session, losing conversation history. 2-hour session expiry is invisible to the frontend.
MEDIUM	database.py:341-343	<code>check_availability</code> computes <code>end_time</code> using <code>date.today()</code> instead of the <code>reservation_date</code> parameter. For future-date bookings, this produces incorrect end times (uses today's date + <code>start_time</code> offset).

MEDIUM	main.py:337-339	get_session generates a new token when the header is missing, but the ChatRequest model also has an optional session_token body field. If both are provided, the header takes precedence but the body field is silently ignored, which could confuse API consumers.
LOW	main.py:183	X-Session-Token is in expose_headers but the frontend reads it from the JSON body. The header exposure is redundant.
LOW	database.py:111-113	TenantConn.__aexit__ releases the connection back to the pool but does not explicitly reset the RLS session variable. While SET LOCAL is transaction-scoped and auto-resets, using TenantConn outside a transaction (which the code never does, but nothing prevents) would leak the RLS context.
LOW	agent.py:573-578	Comment says "FIX: These lines must be indented to stay inside the for loop" but the code is already correctly indented. The comment is misleading and suggests a previous bug was fixed but the comment was left as a marker.
INFO	schema.sql:591	The liftup_app role is granted BYPASSRLS via liftup_admin, which means all RLS policies are bypassed by the application role. This is intentional (commented as "Admin role bypasses RLS") but means RLS provides defense-in-depth only against direct DB access, not application bugs.

6. Prioritized Recommendations

The following recommendations are ordered by urgency. The first three are production-blocking and must be resolved before any 50-restaurant launch. The remaining items are important for long-term stability and security posture.

P0 - Fix Before Launch (Production-Blocking):

1. Refactor modify_reservation to perform cancel+rebook in a single database transaction. This eliminates the TOCTOU race condition that can cause reservation data loss under concurrent load. The book_table logic must be inlined or called within the same transaction context as the cancellation.
2. Replace random.choices() with secrets.choice() in _make_confirmation_code(). This is a one-line fix that eliminates the predictability vulnerability. Also remove the dead code on line 484.

3. Add TLS enforcement middleware that rejects requests without HTTPS when running in production (not DEBUG mode). Tighten CORS to specific domains or require a whitelist configuration per tenant.

P1 - Fix Within 30 Days (Important):

4. Fix the `end_time` calculation in `check_availability` (`database.py` line 341-343) to use `reservation_date` instead of `date.today()`. This causes incorrect availability results for future dates.

5. Add session expiry awareness to the frontend. Either persist the session token in `sessionStorage` or display a warning when the session is likely expired (2-hour window). Consider adding a `"session_expired"` field to the `ChatResponse`.

6. Remove the optional `session_token` field from the `ChatRequest` Pydantic model since it is never read by the handler (the header is the authoritative source). This reduces API surface area and prevents consumer confusion.

P2 - Fix Within 90 Days (Improvements):

7. Add a database-level advisory lock around the `modify_reservation` flow to prevent the cancel-rebook race at the PostgreSQL level, even if the application logic is later refactored.

8. Implement connection pool health checks and automatic reconnection in the `asyncpg` pool configuration. The current setup has no health check interval, meaning stale connections may not be detected until a request fails.

9. Add structured logging with correlation IDs that link the frontend request (visible in the State Inspector) to the backend log entries. This would dramatically improve debugging in production.